

이 프로젝트의 스킬 사용 현황과 설치 안내

두 부분으로 되어 있다.

- 1부 — 이 프로젝트에서 실제로 무슨 스킬이 어떻게 쓰였나 (세션 기록에서 확인한 사실)
- 2부 — 클로드 코드를 막 설치한 학생이 같은 환경을 만드는 방법

1부. 사용 현황

한눈에

이 프로젝트의 웹, 데스크톱 버전 분리 작업은 `superpowers` 플러그인의 스킬 4개로 진행됐다. 세션 기록 (`~/claude/projects/.../*.jsonl`)에서 `Skill` 도구 호출을 세어 확인한 결과다.

순서	스킬	한 일	남은 산출물
1	<code>superpowers:brainstorming</code>	무엇을 만들지 정하고 갈림길마다 대안을 비교	<code>docs/superpowers/specs/2026-08-06-웹-데스크톱-버전-분리-design.md</code>
2	<code>superpowers:writing-plans</code>	설계를 7개 태스크로 나눈 구현 계획서 작성	<code>docs/superpowers/plans/2026-08-06-웹-데스크톱-버전-분리.md</code>
3	<code>superpowers:subagent-driven-development</code>	태스크마다 구현자와 리뷰어를 따로 붙여 실행	커밋 20여 개
4	<code>superpowers:finishing-a-development-branch</code>	최종 리뷰 후 <code>main</code> 에 병합	병합 커밋 <code>63682f0</code>

`docs/superpowers/` 라는 폴더 이름 자체가 이 스킬들이 남긴 흔적이다.

스킬이 실제로 한 일

1. `brainstorming` — 설계 문서를 만들었다

바로 코드를 짜지 않고 먼저 물어본다. 그 결과가 설계 문서의 "결정 기록" 절이다. 다섯 개 갈림길 각각에 대안 표와 선택 근거, 그리고 감수하는 비용이 적혀 있다.

예를 들어 "웹 버전의 추론을 어디서 할 것인가" 에서 브라우저 단독, 파이썬 서버, 단계적 확장 셋을 비교하고 브라우저 단독을 골랐다. 근거는 정적 호스팅 가능성이었고, 감수한 비용은 "전처리를 자바스크립트로 다시 구현하고 그 정확성을 검증해야 한다" 였다.

수업에서 쓸 점: 이 표는 학생에게 "AI 에게 그냥 시키지 말고, 무엇을 포기하는지 함께 적게 하라" 를 보여 주는 실물 예시다.

2. writing-plans — 계획서를 만들었다

설계를 7개 태스크로 나누고, 각 태스크에 검증 방법을 붙였다.

#	태스크	검증
1	폴더 분리 (git mv)	터미널 실행 + .app 더블클릭 회귀
2	가중치 내보내기	파라미터 421,642개, 1,686,568바이트 일치
3	검증 데이터 생성	파이썬 기준 정확도 $\geq 97\%$
4	자바스크립트 순전파	파이썬 확률과 최대 절대차 $\leq 1e-4$
5	자바스크립트 전처리	전체 파이프라인 정확도 $\geq 97\%$
6	웹 앱 화면	직접 그려서 인식 확인
7	CLAUDE.md 세 개	문서에 적은 명령을 실제로 실행

태스크 4와 5를 나눈 것이 핵심이다. 4는 파이썬이 만든 28x28 을 그대로 입력해 순전파만 검사한다. 그래야 5에서 전처리를 붙였을 때 정확도가 떨어지면 원인이 전처리에 있다는 것이 자동으로 확정된다.

수업에서 쓸 점: "검증을 어떻게 설계하면 원인을 좁힐 수 있는가" 의 교과서적 예시다.

3. subagent-driven-development — 구현자와 리뷰어를 갈랐다

이 스킬이 서브에이전트 19회를 부렸다. 구성은 이렇다.

종류	횟수
태스크 구현	7
태스크별 리뷰	8
지적 수정 후 재리뷰	2
최종 전체 브랜치 리뷰	1
최종 리뷰 지적 수정	1
수정분 재리뷰	1

핵심은 구현한 에이전트와 리뷰한 에이전트가 다르다는 것이다. 자기가 짠 코드를 자기가 검토하면 잘 못 찾는다.

이 방식이 실제로 잡아낸 것: 태스크 5에서 `면적평균속소` 의 부동소수 오차로 출력 784개가 전부 `NaN` 이 되어 무엇을 그려도 "0"이라고 답하는 결함이 있었다. 그런데 자동 검증은 정확도 98.0% 로 통과했다. 검증 데이터가 그 경로를 안 밟았기 때문이다. 리뷰어가 숫자를 믿지 않고 코드를 직접 읽어서 찾아냈다.

수업에서 쓸 점: 노션 문서 3.3 의 "9개 미흡한 점" 표가 바로 이 리뷰들의 결과물이다. "검증 통과는 옳다가 아니라 시험해본 범위 안에서는 옳다" 라는 교훈이 여기서 나왔다.

4. `finishing-a-development-branch` — 병합을 마무리했다

작업 브랜치를 `main` 에 병합하고, 미룰 지적과 지금 고칠 지적을 갈랐다. minor 9건은 의도적으로 미뤘다.

쓰지 않은 것

정직하게 밝힌다. 설치는 돼 있으나 이 프로젝트에서 쓰이지 않은 것들이다.

- `superpowers` 의 나머지 스킬 10개 (`test-driven-development` , `systematic-debugging` , `using-git-worktrees` 등). 대화 중 언급은 됐으나 실제 호출 기록은 없다.
- `korean-skills` , `ui-ux-pro-max` , `understand-anything` , `insane-search` , `watch` , `vercel` — 한 번도 호출되지 않았다.

즉 이 프로젝트에 정말로 필요한 것은 `superpowers` 하나뿐이다. 학생에게 7개를 다 깔라고 할 이유가 없다.

노션 문서와의 연결

노션 문서 3.3 의 그림 87 화면 맨 윗줄에 이렇게 찍혀 있다.

```
스킬 실행됨 /superpowers:writing-plans
```

학생이 같은 프롬프트를 넣어도 이 플러그인이 없으면 이 줄이 안 나온다. 설계 문서도 계획서도 생기지 않고, 클로드가 곧장 코드를 짜기 시작한다. 결과물이 문서의 화면과 크게 달라진다.

그래서 2부가 필요하다.

2부. 설치 안내

먼저 알아 둘 것

스킬은 클로드 코드의 기본 기능이 아니다. 누군가 만들어 공개한 플러그인을 설치해야 생긴다. 구조는 이렇다.

마켓플레이스 (플러그인 목록이 있는 깃허브 저장소)

└─ 플러그인 (superpowers)

└─ 스킬 (brainstorming, writing-plans, ...)

그래서 두 단계다. ① 마켓플레이스를 등록하고 ② 그 안의 플러그인을 설치한다.

설치 — 클로드에게 말로 부탁한다

명령을 외울 필요가 없다. 클로드 데스크톱 앱의 대화창에 이렇게 쓰면 된다.

```
superpowers 플러그인을 설치해 줘.
```

클로드가 알아서 마켓플레이스를 등록하고 플러그인을 설치한다. 필요하다면 클로드 코드 CLI 도 먼저 깔아 준다. 운영체제 도 클로드가 가려서 처리하므로 맥이든 윈도우든 같은 말로 된다.

중간에 "이 명령을 실행할까요?" 하고 물으면 허용을 누른다.

설치가 끝나면 클로드 코드를 꺾다가 다시 켜다. 새 세션부터 스킬이 잡힌다. 이미 열려 있던 세션에는 적용되지 않는다. 이 한 가지만 사람이 직접 해야 한다.

세션 안에서 이렇게 확인한다.

```
어떤 스킬을 쓸 수 있어?
```

`superpowers:brainstorming` 같은 이름들이 나오면 성공이다.

직접 명령으로 하고 싶다면

터미널에서 두 줄이다.

```
claude plugin marketplace add obra/superpowers
```

```
claude plugin install superpowers@superpowers-dev
```

`obra/superpowers` 는 깃허브 저장소 주소다(<https://github.com/obra/superpowers>). `superpowers-dev` 는 그 저장소가 스스로 붙인 마켓플레이스 이름이라 저장소 이름과 다르다. 헷갈리기 쉬운 지점이다.

설치 확인은 이렇게 한다.

```
claude plugin list
```

목록에 `superpowers` 가 보이면 된다.

`claude` 명령이 없다면 클로드 코드 CLI 가 설치돼 있지 않은 것이다. 데스크톱 앱을 깔아도 CLI 는 함께 오지 않는다. 확인한 사실이다 — `brew install --cask claude` 가 설치하는 것은 `Claude.app` 하나뿐이고, CLI 는 `claude-code` 라는 별개 cask 다. 맥에서는 아래 한 줄로 깐다.

```
brew install --cask claude-code
```

노션 문서 3.1.2 의 CLI 절은 `npm install -g @anthropic-ai/claude-code` 를 쓰는데, 그러려면 노드JS 가 필요하다. 이 수업은 노드JS 를 쓰지 않기로 했으므로(노션 문서 3.4.1 참고) 위의 Homebrew 방식을 권한다.

세션 안에서는 이렇게 확인한다.

어떤 스킬을 쓸 수 있어?

`superpowers:brainstorming` 같은 이름들이 나오면 성공이다.

써 보기

설치했다고 자동으로 도는 것이 아니다. 부르거나, 부를 만한 일을 시켜야 한다.

직접 부르는 방법이다.

```
/superpowers:brainstorming
```

또는 그냥 말로 시켜도 된다. 만들기 전에 설계부터 하라는 뜻이 담긴 요청이면 클로드가 알아서 집는다.

손글씨 인식 프로그램을 웹 버전과 데스크톱 버전으로 나누고 싶어.

바로 코드 짜지 말고, 먼저 설계와 계획을 세워 줘.

화면 위쪽에 스킬 실행됨 `/superpowers:brainstorming` 이 뜨면 제대로 걸린 것이다.

지우기

```
claude plugin uninstall superpowers@superpowers-dev
```

자주 막히는 곳

증상	원인과 해결
<code>claude: command not found</code>	CLI 미설치. 데스크톱 앱을 깔아도 CLI 는 안 따라온다. <code>brew install --cask claude-code</code>
설치는 됐는데 스킬이 안 보임	클로드 코드를 재시작하지 않았다. 껏다 켜다
<code>plugin not found</code>	마켓플레이스 등록을 건너뛰었다. <code>marketplace add</code> 를 먼저
<code>superpowers@superpowers</code> 로 썼는데 안 됨	마켓플레이스 이름은 <code>superpowers-dev</code> 다. 저장소 이름과 다르다
스킬이 있는데 안 걸림	요청이 너무 짧으면 클로드가 그냥 코드를 짤다. "설계부터", "계획을 세워 줘" 를 넣는다

수업 운영 제언

학생에게 7개를 다 깔게 하지 않는 것을 권한다. 이 프로젝트에 실제로 쓰인 것은 `superpowers` 뿐이고, 플러그인이 많을 수록 시작이 느려지고 설치 단계에서 이탈이 생긴다.

한 가지 판단이 필요한 점을 밝힌다. `superpowers` 를 깔면 3장 실습의 진행 방식이 달라진다. 클로드가 바로 코드를 짜 지 않고 설계와 계획을 먼저 만들기 때문에, 노션 문서 3.2 의 "프롬프트 한 줄 → 앱 완성" 흐름과 어긋난다. 그래서 이런 배치를 제안한다.

절	스킬	이유
3.2 (첫 실습)	설치하지 않은 상태	"AI 에게 시키면 바로 만들어진다" 는 첫 경험이 살아 있어야 한다
3.3 (웹 버전)	여기서 설치	규모가 커지면 왜 설계와 리뷰가 필요한지 학생이 몸으로 느낀 다음이다

이 순서면 스킬이 "좋다니까 까는 것" 이 아니라 "막혀 봤으니 필요한 것" 이 된다.

확인한 것과 확인하지 못한 것

확인한 것

- 스킬 호출 4건과 서브에이전트 19건은 이 프로젝트의 세션 기록 파일 4개를 직접 세어 확인했다.
- 설치된 플러그인 7개와 마켓플레이스 주소는 `~/.claude/plugins/installed_plugins.json` , `known_marketplaces.json` 에서 확인했다.
- 마켓플레이스 이름이 `superpowers-dev` 라는 것은 해당 저장소의 `marketplace.json` 에서 확인했다.
- `claude plugin marketplace add` , `claude plugin install` 의 문법은 `--help` 로 확인했다.

확인하지 못한 것

- 빈 컴퓨터에서 이 설치 절차를 처음부터 끝까지 돌려 보지는 않았다. 이미 설치된 환경에서 역으로 확인한 것이다. 수업 전에 한 대에서 실제로 봐야 보기를 권한다.
- 데스크톱 앱 안에서 `/plugin` 화면으로도 설치가 되는지는 확인하지 않았다. 말로 부탁하는 방식이 있으므로 급하지 않다.
- 윈도우용 CLI 설치 명령은 확인하지 않았다. 말로 부탁하면 클로드가 가려서 처리하므로 이 문서에 적지 않았다.

2026-08-08 에 고친 것

이 문서의 앞 판에는 사실과 다른 안내가 있었다. 밝혀 둔다.

앞 판	실제
"데스크톱 앱만 깔아도 CLI 가 보통 함께 들어온다"	아니다. <code>brew install --cask claude</code> 의 설치 항목은 <code>Claude.app</code> 하나뿐이고, CLI 는 <code>claude-code</code> 라는 별개 cask 다. <code>brew info</code> 로 두 cask 를 대조해 확인했다
대안으로 <code>npm install -g @anthropic-ai/claude-code</code> 를 안내	그러려면 노드JS 가 필요한데, 이 수업은 노드JS 를 쓰지 않기로 했다(노션 3.4.1). 맥은 <code>brew install --cask claude-code</code> 로 바꿨다
설치를 "터미널에서 두 줄" 로 안내	<code>claude plugin ...</code> 은 슬래시 명령이 아니라 셸 명령이라 클로드가 대신 실행할 수 있다. 세션 안에서 <code>claude plugin list</code> 가 도는 것, <code>install</code> 과 <code>marketplace add</code> 에 대화형 확인이 없는 것, <code>/opt/homebrew/bin</code> 에 쓰기 권한이 있어 <code>sudo</code> 가 필요 없는 것을 확인했다. 그래서 말로 부탁하는 방식을 앞에 두었다